



Enunciado de la Prueba de desempeño

Módulo 5.1 - Java

Java SE + JOptionPane + JDBC + Capas + Archivos + Excepciones + Pruebas

Caso de uso:

HotelNova, una cadena de hospedajes que administra reservas, habitaciones y huéspedes, ha venido gestionando su información mediante hojas de cálculo y registros manuales. Esto ha ocasionado diversos problemas operativos:

- Duplicidad de reservas debido a errores en el manejo de identificadores.
- Inconsistencias en la disponibilidad de habitaciones.
- Falta de control sobre los usuarios y roles del sistema.
- Registros de check-in y check-out incompletos o con errores.
- Pérdida de información por el uso de archivos compartidos sin respaldo.

La dirección de HotelNova ha decidido desarrollar un sistema interno en Java SE con interfaz gráfica simple mediante JOptionPane, que permita gestionar de forma eficiente las habitaciones, los huéspedes, los usuarios y las reservas. Este sistema debe implementarse aplicando buenas prácticas de arquitectura por capas, manejo de excepciones, persistencia con JDBC, exportación de datos a archivos y pruebas unitarias.

Objetivo:

- Construir una aplicación **Java SE (versión 17 o superior)** utilizando **JOptionPane para la interfaz de usuario**, **JDBC (MySQL o PostgreSQL)** para la persistencia de datos, y una **arquitectura por capas (controller, service, dao, model)**.
- El sistema debe incluir **manejo de archivos (config y exportación CSV)**, **excepciones personalizadas** y **pruebas con JUnit 5**, con el propósito de garantizar la **integridad, consistencia y trazabilidad** de la información.
- El modelado de clases, la autenticación básica y las operaciones CRUD forman parte del alcance del ejercicio.

El sistema deberá:

- Centralizar la información de habitaciones, huéspedes, usuarios y reservas en una sola aplicación.
- Facilitar la gestión de habitaciones (crear, editar, activar/desactivar, filtrar por tipo o precio por noche).



- Automatizar procesos de inicio de sesión, registro de huéspedes/usuarios con propiedades por defecto, y registrar operaciones CRUD simulando trazas de “llamadas HTTP” en consola o logs.
- Aplicar principios de **modularidad, separación de capas y validaciones de negocio** (número de habitación único, disponibilidad de habitaciones, huéspedes activos, fechas de reserva válidas).
- Implementar transacciones JDBC para garantizar coherencia en reservas, check-in y check-out.
- Permitir **exportar información a archivos CSV** y leer parámetros de configuración desde un archivo .properties.
- Ejecutar **pruebas unitarias** para validar las reglas de negocio y el correcto funcionamiento de los servicios.

Funcionalidades principales

Para alcanzar un resultado óptimo en esta prueba, deberás cumplir cada uno de los siguientes requisitos y funcionalidades:

Requisitos:

1) Gestión de Habitaciones

- Registrar nuevas habitaciones con los campos mínimos: **id, numero, tipo, capacidad, precioPorNoche, estado (DISPONIBLE/OCUPADA), isActiva, createdAt**.
- Editar/actualizar información de habitaciones existentes.
- Validar que el número de habitación sea único antes de registrar.
- Listar habitaciones y filtrar por tipo o estado.
- Mostrar los resultados en tablas de texto dentro de JOptionPane.

2) Gestión de Usuarios y Autenticación

- Implementar inicio de sesión (login) con validación de credenciales y roles (ADMIN / RECEPCIONISTA).
- Las contraseñas deben almacenarse de forma segura utilizando un mecanismo de encriptación o hash (por ejemplo, BCrypt), evitando guardar contraseñas en texto plano.
- Registrar logs en consola simulando llamadas HTTP (GET/POST/PATCH/DELETE).

3) Interfaz gráfica (JOptionPane)

- Construir menús modales: **Habitaciones, Huéspedes, Usuarios, Reservas, Exportaciones**.

- Mostrar confirmaciones y mensajes de éxito/error con `showMessageDialog`.
- Crear helpers para imprimir listados en formato de tabla (columnas alineadas, etiquetas [ACTIVO]/[INACTIVO], [DISPONIBLE]/[OCUPADA]).

4) CRUD + JDBC + Transacciones

- Definir interfaces DAO.
- Implementar clases JDBC con consultas SQL (SELECT, INSERT, UPDATE, DELETE).
- Usar **transacciones**:
 - **Reserva (check-in)** → `setAutoCommit(false)` → insertar reserva → actualizar estado de habitación (OCUPADA) → `commit()` / `rollback()`.
 - **Check-out (salida)** → actualizar reserva (finalizada) → calcular costo total de la estadía ($\text{noches} \times \text{precioPorNoche} + \text{impuesto IVA definido en config.properties}$) → actualizar estado de habitación (DISPONIBLE) → `commit()`.
- Usar try-with-resources para cerrar conexiones y liberar recursos.

5) Manejo de Archivos

- Crear archivo `config.properties` con parámetros:

```
db.url=jdbc:<dbms>://<host>:<port>/<database>
db.user=<username>
db.password=<password>
horaCheckIn=15
horaCheckOut=12
iva=0.19
```

- Exportar datos a **CSV**:
 - `habitaciones_export.csv` (listado completo de habitaciones).
 - `reservas_activas.csv` (solo reservas en curso o activas).
- Registrar actividad y errores en `app.log` con `java.util.logging` o similar.

6) Excepciones y Validaciones

- Crear excepciones personalizadas.
- Capturar errores con try/catch.
- Validar reglas de negocio:
 - Número de habitación único.
 - Habitación disponible para la reserva.
 - Huésped activo.
 - Fechas de reserva válidas (check-in < check-out).
 - No permitir reservas solapadas para la misma habitación.

- No permitir check-out sin un check-in previo.
- Mostrar errores en JOptionPane y guardar detalles en el log.

7) Pruebas (JUnit 5)

- Escribir pruebas unitarias para los servicios principales:
 - Validación de número de habitación único al registrar.
 - Validación de disponibilidad de habitaciones antes de realizar una reserva.
 - Validación de huésped activo para permitir la reserva.
 - Validación de fechas de reserva (check-in debe ser menor que check-out).
 - Validación de no solapamiento de reservas para una misma habitación.
 - Validación de flujo de check-out (no permitir salida sin reserva activa).
 - Cálculo del costo de la estadía (noches × precio por noche, más el impuesto (IVA) definido en el archivo config.properties).
- Utilizar assertEquals, assertThrows, assertTrue.
- Ejecutar pruebas con mvn test.

Criterios de aceptación

1. Gestión de Habitaciones

- a. Se puede registrar y editar una habitación.
- b. El sistema valida número de habitación único.
- c. Se listan y filtran habitaciones correctamente.

2. Usuarios y Autenticación

- a. Login funcional con roles.
- b. CRUD de usuarios operativo.
- c. Las contraseñas se almacenan de forma segura (hash, no texto plano).

3. Interfaz (JOptionPane)

- a. Menús claros, confirmaciones, listados legibles.

4. JDBC y Transacciones

- a. DAO y servicios ejecutan CRUD correctamente.
- b. Reserva (check-in) y check-out se procesan de forma individual y consistente.

5. Archivos y Logs

- a. Configuración leída desde .properties.
- b. Exportaciones CSV correctas.
- c. Logs funcionales.

6. Excepciones y Validaciones

- a. Manejo claro de errores y mensajes al usuario.
- b. Validaciones de negocio respetadas.

7. Documentación

- a. README.md con:
 - i. Descripción general del sistema.
 - ii. Requisitos previos (Java, Maven, DB).



- iii. Pasos de configuración y ejecución.
- iv. Capturas de pantalla de JOptionPane.
- v. Diagramas de Clases.
- vi. Diagrama de Casos de Uso.

Entregables

- Enlace al repositorio GitHub (público).
- Proyecto comprimido (.zip).
- README.md con instrucciones y datos del Coder (Nombre, Clan, correo, documento).